Problem Statements Report

Generated on: 2026-03-08 22:09

Week - 4: Deque

Q1. Implement Deque using Array

MEDIUM CODING

In this exercise, you must sequentially implement a Deque (Double-Ended Queue) utilizing a fixed-size array. You are required to support dynamic insertions and deletions from both the front and the rear of the data structure. You need to maintain 'front' and 'rear' pointers and precisely handle both Underflow and Overflow anomalies, while treating the array as completely linear or circularly buffered logic if bounds are exceeded.

Input Format: An integer N (number of operations), followed by N lines of commands:
1. 'insert_front X': Add integer X to the front.
2. 'insert_rear X': Add integer X to the rear.
3. 'delete_front': Remove and print the front element.
4. 'delete_rear': Remove and print the rear element.
5. 'display': Print all elements from front to rear.

Output Format: For 'delete_front' and 'delete_rear', print the value or 'Underflow'. For 'insert_front' and 'insert_rear', print 'Overflow' if full. For 'display', print elements separated by space or 'Empty'.

Constraints: *Array Capacity = 100. 1 <= N <= 200.*

Hints: *Initialize 'front' and 'rear' to -1. Handle cases where the queue gets full or empty by resetting the pointers appropriately. For a robust structure, use circular modulus logic or boundaries wrap-around checks (0 to 99).*

Test Cases:

Sample Test Case #1

Input:

```
4
insert_rear 10
insert_front 20
display
delete_front
```

Expected Output:

```
20 10
20
```

Sample Test Case #2

Input:

```
2
delete_front
delete_rear
```

Expected Output:

```
Underflow
Underflow
```

Sample Test Case #3

Input:

```
3
insert_front 5
display
delete_rear
```

Expected Output:

```
5
5
```

Solution Strategy:

A robust array-based Deque must handle wraparound boundaries. When inserting at the front at index 0, the next location logically wraps back to index 99. The implementation effectively checks for array exhaustion via boundary checks.

Reference Solution:

```
#include <stdio.h>
#include <string.h>
int dq[100], f=-1, r=-1, n, v; char op[20];
int main() {
    if(scanf("%d", &n) != 1) return 0;
    while(n--) {
        scanf("%s", op);
        if(strcmp(op, "insert_front") == 0) {
            if(scanf("%d", &v) == 1) {
                if((f == 0 && r == 99) || f == r + 1) { printf("Overflow\n"); }
                else { if(f == -1) f = r = 0; else if(f == 0) f = 99; else f--; dq[f] = v; }
            }
        } else if(strcmp(op, "insert_rear") == 0) {
            if(scanf("%d", &v) == 1) {
                if((f == 0 && r == 99) || f == r + 1) { printf("Overflow\n"); }
                else { if(f == -1) f = r = 0; else if(r == 99) r = 0; else r++; dq[r] = v; }
            }
        } else if(strcmp(op, "delete_front") == 0) {
            if(f == -1) printf("Underflow\n");
            else { printf("%d\n", dq[f]); if(f == r) f = r = -1; else if(f == 99) f = 0; else f
    ++; }
        } else if(strcmp(op, "delete_rear") == 0) {
            if(f == -1) printf("Underflow\n");
            else { printf("%d\n", dq[r]); if(f == r) f = r = -1; else if(r == 0) r = 99; else r
    --; }
        } else if(strcmp(op, "display") == 0) {
            if(f == -1) printf("Empty\n");
            else { int i = f; while(1){ printf("%d%c", dq[i], i==r?'\n':' '); if(i==r)break; if
    (i==99)i=0; else i++; } }
        }
    }
    return 0;
}
```

Q2. Palindrome Checker using Deque

EASY CODING

A palindrome is a word or string that reads identical from front-to-back as back-to-front. You must utilize a Deque format to verify this. Your program should sequentially insert all characters of an inputted string into the rear of a deque data structure. To evaluate, repeatedly delete and compare a character extracted from the front against a character extracted from the rear. Any mismatch immediately invalidates the palindrome condition.

Input Format:	A single string S without spaces.
Output Format:	Print 'Palindrome' if the given string is a palindrome, otherwise print 'Not Palindrome'.
Constraints:	*1 <= length of string <= 1000.*
Hints:	*Loop and push all characters into the deque array incrementing the rear. Iterate from the front and rear sequentially bridging inwards until pointers cross to check pairs.*

Test Cases:

Sample Test Case #1

Input:

```
radar
```

Expected Output:

```
Palindrome
```

Sample Test Case #2

Input:

```
hello
```

Expected Output:

```
Not Palindrome
```

Sample Test Case #3

Input:

```
a
```

Expected Output:

```
Palindrome
```

Solution Strategy:

By putting elements in a Double-Ended Queue (simulated via array bounds), we have direct simultaneous access to the First matching the Last element sequentially until we bridge completely towards the center.

Reference Solution:

```c
#include <stdio.h>
#include <string.h>
int main() {
    char s[1001], dq[1001]; int f = 0, r = -1;
    if(scanf("%s", s) == 1) {
        for(int i=0; s[i]; i++) dq[++r] = s[i];
        int isPal = 1;
        while(f < r) {
            if(dq[f++] != dq[r--]) { isPal = 0; break; }
        }
        if(isPal) printf("Palindrome\n");
        else printf("Not Palindrome\n");
    }
    return 0;
}
```

Week 5: Singly Linked List

Q1. Basic Operations and Traversal

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Create a Singly Linked List, Display all elements of the list, Count the number of nodes.

Input Format: Integer N (number of operations), followed by N lines of structural commands (e.g. 'ins_end X', 'display', 'count').

Output Format: For 'display', print space-separated elements from head to tail or 'Empty'. For 'count', print the integer count.

Constraints: *0 <= N <= 1000. Nodes must be dynamically allocated.*

Hints: *Maintain a head pointer initialized to NULL. For counting, traverse iteratively from head until you hit NULL.*

Test Cases:

Sample Test Case #1

Input:

```
3
ins_end 10
ins_end 20
display
```

Expected Output:

```
10 20
```

Sample Test Case #2

Input:

```
4
ins_end 5
ins_end 15
count
length
```

Expected Output:

```
2
2
```

Sample Test Case #3

Input:

```
1
display
```

Expected Output:

```
Empty
```

Solution Strategy:

Basic list operations rely on the standard pointer-chasing technique. Operations like count and display take O(N) time as

they require visiting every node.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=head; head=nn; }
void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void ip(int p, int v) { if(p==1) ib(v); else { Node* t=head; for(int i=1; i<p-1 && t; i++) t=t-
    >n; if(t) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn; } } }
void db() { if(head) { Node* t=head; head=head->n; free(t); } else printf("Underflow\n"); }
void de() { if(!head) printf("Underflow\n"); else if(!head->n) { free(head); head=NULL; } else
    { Node* t=head; while(t->n->n) t=t->n; free(t->n); t->n=NULL; } }
void dp(int p) { if(!head) printf("Underflow\n"); else if(p==1) db(); else { Node* t=head; for(
    int i=1; i<p-1 && t->n; i++) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } e
    lse printf("Underflow\n"); } }
void dk(int v) { if(!head) printf("Underflow\n"); else if(head->d==v) db(); else { Node* t=head
    ; while(t->n && t->n->d!=v) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } el
    se printf("Not Found\n"); } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d,
     t->n?' ':'\n'); t=t->n; } } }
void cnt() { int c=0; Node* t=head; while(t) { c++; t=t->n; } printf("%d\n", c); }
void sch(int v) { int p=1; Node* t=head; while(t) { if(t->d==v) { printf("Found at %d\n", p); r
    eturn; } p++; t=t->n; } printf("Not Found\n"); }
void rev() { Node *p=NULL, *c=head, *nx=NULL; while(c){ nx=c->n; c->n=p; p=c; c=nx; } head=p; }
void mid() { if(!head){ printf("Empty\n"); return; } Node *s=head, *f=head; while(f && f->n){ s
    =s->n; f=f->n->n; } printf("Middle: %d\n", s->d); }

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
        else if(!strcmp(op, "reverse")) rev();
        else if(!strcmp(op, "middle")) mid();
    }
    return 0;
}
```

Q2. Search Operation

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Search for the presence and position of an element.

Input Format: Integer N, followed by N operations including 'ins_end X' and 'search X'.

Output Format: For 'search X', print 'Found at P' where P is the 1-based traversal index, or 'Not Found'.

Constraints: *0 <= N <= 1000. Nodes must be dynamically allocated.*

Hints: *Iterate using a while loop and a counter variable. Break early and return the counter if node->data == target.*

Test Cases:

Sample Test Case #1

Input:

```
4
ins_end 10
ins_end 20
search 20
search 30
```

Expected Output:

```
Found at 2
Not Found
```

Sample Test Case #2

Input:

```
1
search 5
```

Expected Output:

```
Not Found
```

Sample Test Case #3

Input:

```
3
ins_end 1
ins_end 2
search 1
```

Expected Output:

```
Found at 1
```

Solution Strategy:

Searching is an O(N) operation in a sequentially accessed linked list. Breaking the loop upon match provides best-case performance optimization.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=head; head=nn; }
void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void ip(int p, int v) { if(p==1) ib(v); else { Node* t=head; for(int i=1; i<p-1 && t; i++) t=t-
    >n; if(t) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn; } } }
void db() { if(head) { Node* t=head; head=head->n; free(t); } else printf("Underflow\n"); }
void de() { if(!head) printf("Underflow\n"); else if(!head->n) { free(head); head=NULL; } else
    { Node* t=head; while(t->n->n) t=t->n; free(t->n); t->n=NULL; } }
void dp(int p) { if(!head) printf("Underflow\n"); else if(p==1) db(); else { Node* t=head; for(
    int i=1; i<p-1 && t->n; i++) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } e
    lse printf("Underflow\n"); } }
void dk(int v) { if(!head) printf("Underflow\n"); else if(head->d==v) db(); else { Node* t=head
    ; while(t->n && t->n->d!=v) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } el
    se printf("Not Found\n"); } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d,
     t->n?' ':'\n'); t=t->n; } } }
void cnt() { int c=0; Node* t=head; while(t) { c++; t=t->n; } printf("%d\n", c); }
void sch(int v) { int p=1; Node* t=head; while(t) { if(t->d==v) { printf("Found at %d\n", p); r
    eturn; } p++; t=t->n; } printf("Not Found\n"); }
void rev() { Node *p=NULL, *c=head, *nx=NULL; while(c){ nx=c->n; c->n=p; p=c; c=nx; } head=p; }
void mid() { if(!head){ printf("Empty\n"); return; } Node *s=head, *f=head; while(f && f->n){ s
    =s->n; f=f->n->n; } printf("Middle: %d\n", s->d); }

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
        else if(!strcmp(op, "reverse")) rev();
        else if(!strcmp(op, "middle")) mid();
    }
    return 0;
}
```

Q3. Insertion Operations

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Insert at Beginning, Insert at End, Insert at Given Position.

Input Format: Integer N, followed by inserting operations 'ins_beg X', 'ins_end X', 'ins_pos P X'.

Output Format: No direct output for insertion. Verify the memory structural state using a sequential 'display' command.

Constraints: *0 <= N <= 1000. Nodes must be dynamically allocated.*

Hints: *When inserting at position P (P > 1), navigate exactly to position P-1 and carefully reroute the 'next' pointer.*

Test Cases:

Sample Test Case #1

Input:

```
5
ins_end 10
ins_beg 5
ins_end 20
ins_pos 2 8
display
```

Expected Output:

```
5 8 10 20
```

Sample Test Case #2

Input:

```
2
ins_end 5
display
```

Expected Output:

```
5
```

Sample Test Case #3

Input:

```
3
ins_end 10
ins_pos 1 5
display
```

Expected Output:

```
5 10
```

Solution Strategy:

Insertion manipulates memory node links. Prepending is an O(1) operation, whereas arbitrary position insertion requires O(N) traversal overhead.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=head; head=nn; }
void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void ip(int p, int v) { if(p==1) ib(v); else { Node* t=head; for(int i=1; i<p-1 && t; i++) t=t-
    >n; if(t) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn; } } }
void db() { if(head) { Node* t=head; head=head->n; free(t); } else printf("Underflow\n"); }
void de() { if(!head) printf("Underflow\n"); else if(!head->n) { free(head); head=NULL; } else
    { Node* t=head; while(t->n->n) t=t->n; free(t->n); t->n=NULL; } }
void dp(int p) { if(!head) printf("Underflow\n"); else if(p==1) db(); else { Node* t=head; for(
    int i=1; i<p-1 && t->n; i++) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } e
    lse printf("Underflow\n"); } }
void dk(int v) { if(!head) printf("Underflow\n"); else if(head->d==v) db(); else { Node* t=head
    ; while(t->n && t->n->d!=v) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } el
    se printf("Not Found\n"); } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d,
     t->n?' ':'\n'); t=t->n; } } }
void cnt() { int c=0; Node* t=head; while(t) { c++; t=t->n; } printf("%d\n", c); }
void sch(int v) { int p=1; Node* t=head; while(t) { if(t->d==v) { printf("Found at %d\n", p); r
    eturn; } p++; t=t->n; } printf("Not Found\n"); }
void rev() { Node *p=NULL, *c=head, *nx=NULL; while(c){ nx=c->n; c->n=p; p=c; c=nx; } head=p; }
void mid() { if(!head){ printf("Empty\n"); return; } Node *s=head, *f=head; while(f && f->n){ s
    =s->n; f=f->n->n; } printf("Middle: %d\n", s->d); }

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
        else if(!strcmp(op, "reverse")) rev();
        else if(!strcmp(op, "middle")) mid();
    }
    return 0;
}
```

Q4. Deletion Operations

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Delete at Beginning, Delete at End, Delete at position, Delete by Key.

Input Format: Integer N, followed by N operations like 'del_beg', 'del_end', 'del_pos P', 'del_key X'.

Output Format: Print 'Underflow' if attempting deletion from an empty list. Print 'Not Found' if deleting an absent key.

Constraints: *0 <= N <= 1000. Nodes must be dynamically allocated.*

Hints: *Ensure robust NULL checks before de-referencing. Do not forget to use 'free()' to prevent critical memory leaks.*

Test Cases:

Sample Test Case #1

Input:

```
7
ins_end 10
ins_end 20
ins_end 30
del_beg
del_end
del_key 20
display
```

Expected Output:

```
Empty
```

Sample Test Case #2

Input:

```
2
del_beg
display
```

Expected Output:

```
Underflow
Empty
```

Sample Test Case #3

Input:

```
3
ins_end 10
del_pos 1
display
```

Expected Output:

```
Empty
```

Solution Strategy:

A robust deletion engine safely bypasses the doomed node logically by updating the predecessor's next pointer,

sequentially executing a free() operation.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=head; head=nn; }
void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void ip(int p, int v) { if(p==1) ib(v); else { Node* t=head; for(int i=1; i<p-1 && t; i++) t=t-
    >n; if(t) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn; } } }
void db() { if(head) { Node* t=head; head=head->n; free(t); } else printf("Underflow\n"); }
void de() { if(!head) printf("Underflow\n"); else if(!head->n) { free(head); head=NULL; } else
    { Node* t=head; while(t->n->n) t=t->n; free(t->n); t->n=NULL; } }
void dp(int p) { if(!head) printf("Underflow\n"); else if(p==1) db(); else { Node* t=head; for(
    int i=1; i<p-1 && t->n; i++) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } e
    lse printf("Underflow\n"); } }
void dk(int v) { if(!head) printf("Underflow\n"); else if(head->d==v) db(); else { Node* t=head
    ; while(t->n && t->n->d!=v) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } el
    se printf("Not Found\n"); } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d,
     t->n?' ':'\n'); t=t->n; } } }
void cnt() { int c=0; Node* t=head; while(t) { c++; t=t->n; } printf("%d\n", c); }
void sch(int v) { int p=1; Node* t=head; while(t) { if(t->d==v) { printf("Found at %d\n", p); r
    eturn; } p++; t=t->n; } printf("Not Found\n"); }
void rev() { Node *p=NULL, *c=head, *nx=NULL; while(c){ nx=c->n; c->n=p; p=c; c=nx; } head=p; }
void mid() { if(!head){ printf("Empty\n"); return; } Node *s=head, *f=head; while(f && f->n){ s
    =s->n; f=f->n->n; } printf("Middle: %d\n", s->d); }

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
        else if(!strcmp(op, "reverse")) rev();
        else if(!strcmp(op, "middle")) mid();
    }
    return 0;
}
```

Q5. List Reversal

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Reverse a structurally intact Linked List in-place.

Input Format: Integer N, followed by typical insertions and the exact command 'reverse'.

Output Format: Use 'display' after reversal to verify order. No explicit printed output from 'reverse'.

Constraints: *0 <= N <= 1000. Nodes must be dynamically allocated.*

Hints: *Three sliding pointers (prev, current, next) gracefully flip the 'next' orientation iteratively.*

Test Cases:

Sample Test Case #1

Input:

```
4
ins_end 1
ins_end 2
reverse
display
```

Expected Output:

```
2 1
```

Sample Test Case #2

Input:

```
2
reverse
display
```

Expected Output:

```
Empty
```

Sample Test Case #3

Input:

```
3
ins_end 10
reverse
display
```

Expected Output:

```
10
```

Solution Strategy:

An efficient in-place reversal flips the linkage directions iteratively in exactly O(N) time without requiring duplicated structural memory allocation.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=head; head=nn; }
void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void ip(int p, int v) { if(p==1) ib(v); else { Node* t=head; for(int i=1; i<p-1 && t; i++) t=t-
    >n; if(t) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn; } } }
void db() { if(head) { Node* t=head; head=head->n; free(t); } else printf("Underflow\n"); }
void de() { if(!head) printf("Underflow\n"); else if(!head->n) { free(head); head=NULL; } else
    { Node* t=head; while(t->n->n) t=t->n; free(t->n); t->n=NULL; } }
void dp(int p) { if(!head) printf("Underflow\n"); else if(p==1) db(); else { Node* t=head; for(
    int i=1; i<p-1 && t->n; i++) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } e
    lse printf("Underflow\n"); } }
void dk(int v) { if(!head) printf("Underflow\n"); else if(head->d==v) db(); else { Node* t=head
    ; while(t->n && t->n->d!=v) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } el
    se printf("Not Found\n"); } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d,
     t->n?' ':'\n'); t=t->n; } } }
void cnt() { int c=0; Node* t=head; while(t) { c++; t=t->n; } printf("%d\n", c); }
void sch(int v) { int p=1; Node* t=head; while(t) { if(t->d==v) { printf("Found at %d\n", p); r
    eturn; } p++; t=t->n; } printf("Not Found\n"); }
void rev() { Node *p=NULL, *c=head, *nx=NULL; while(c){ nx=c->n; c->n=p; p=c; c=nx; } head=p; }
void mid() { if(!head){ printf("Empty\n"); return; } Node *s=head, *f=head; while(f && f->n){ s
    =s->n; f=f->n->n; } printf("Middle: %d\n", s->d); }

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
        else if(!strcmp(op, "reverse")) rev();
        else if(!strcmp(op, "middle")) mid();
    }
    return 0;
}
```

Q6. Middle Element Detection

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Find the exact Middle Element of an uneven list systematically.

Input Format: Integer N, operations, and 'middle' to trigger the check.

Output Format: Print 'Middle: X' where X is the middle element data. Print 'Empty' if the link asserts NULL natively.

Constraints: *0 <= N <= 1000. Nodes must be dynamically allocated.*

Hints: *Tortoise and Hare algorithm dynamically iterates 'slow' by 1 step and 'fast' by 2 steps to triangulate the midpoint.*

Test Cases:

Sample Test Case #1

Input:

```
6
ins_end 1
ins_end 2
ins_end 3
ins_end 4
ins_end 5
middle
```

Expected Output:

```
Middle: 3
```

Sample Test Case #2

Input:

```
1
middle
```

Expected Output:

```
Empty
```

Sample Test Case #3

Input:

```
5
ins_end 1
ins_end 2
ins_end 3
ins_end 4
middle
```

Expected Output:

```
Middle: 3
```

Solution Strategy:

The Tortoise and Hare algorithm perfectly identifies the midpoint without knowing the length explicitly, operating inside a single optimized O(N/2) loop.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=head; head=nn; }
void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void ip(int p, int v) { if(p==1) ib(v); else { Node* t=head; for(int i=1; i<p-1 && t; i++) t=t-
    >n; if(t) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn; } } }
void db() { if(head) { Node* t=head; head=head->n; free(t); } else printf("Underflow\n"); }
void de() { if(!head) printf("Underflow\n"); else if(!head->n) { free(head); head=NULL; } else
    { Node* t=head; while(t->n->n) t=t->n; free(t->n); t->n=NULL; } }
void dp(int p) { if(!head) printf("Underflow\n"); else if(p==1) db(); else { Node* t=head; for(
    int i=1; i<p-1 && t->n; i++) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } e
    lse printf("Underflow\n"); } }
void dk(int v) { if(!head) printf("Underflow\n"); else if(head->d==v) db(); else { Node* t=head
    ; while(t->n && t->n->d!=v) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } el
    se printf("Not Found\n"); } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d,
     t->n?' ':'\n'); t=t->n; } } }
void cnt() { int c=0; Node* t=head; while(t) { c++; t=t->n; } printf("%d\n", c); }
void sch(int v) { int p=1; Node* t=head; while(t) { if(t->d==v) { printf("Found at %d\n", p); r
    eturn; } p++; t=t->n; } printf("Not Found\n"); }
void rev() { Node *p=NULL, *c=head, *nx=NULL; while(c){ nx=c->n; c->n=p; p=c; c=nx; } head=p; }
void mid() { if(!head){ printf("Empty\n"); return; } Node *s=head, *f=head; while(f && f->n){ s
    =s->n; f=f->n->n; } printf("Middle: %d\n", s->d); }

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
        else if(!strcmp(op, "reverse")) rev();
        else if(!strcmp(op, "middle")) mid();
    }
    return 0;
}
```

Q7. Sort Linked List

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Sort an explicitly provided random Linked List sequentially.

Input Format: Integer N specifying the total list size, exactly followed by N integers. Input 0 signifies 'Empty'.

Output Format: Print securely strictly sorted ascending values, space delineated.

Constraints: *0 <= N <= 1000. Nodes must be dynamically allocated.*

Hints: *For singly linked structures, swapping integer values iteratively inside nested loops is structurally simpler than unlinking nodes.*

Test Cases:

Sample Test Case #1

Input:

```
5
4 2 5 1 3
```

Expected Output:

```
1 2 3 4 5
```

Sample Test Case #2

Input:

```
3
10 10 10
```

Expected Output:

```
10 10 10
```

Sample Test Case #3

Input:

```
0
```

Expected Output:

```
Empty
```

Solution Strategy:

A bubble-sort mechanism directly applied swaps the dynamic payload payloads. O(N^2) complexity provides an implementable yet safe structural sorting protocol.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* create(int n) { if(n<=0) return NULL; Node *h=NULL, *t=NULL; while(n--) { int v; scanf("%
    d", &v); Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!h) h=t=nn; else { t->n=nn;
     t=nn; } } return h; }
void sort(Node* h) { for(Node* i=h; i; i=i->n) for(Node* j=i->n; j; j=j->n) if(i->d > j->d){ in
    t t=i->d; i->d=j->d; j->d=t; } }
int main() {
    int n; if(scanf("%d",&n)!=1) return 0; Node* h1=create(n);
    sort(h1);
    Node* x = h1;
    if(!x) printf("Empty\n");
    while(x) { printf("%d%c", x->d, x->n?' ':'\n'); x=x->n; }
    return 0;
}
```

Q8. Merge Two Sorted Linked Lists

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Merge Two specifically pre-Sorted Linked Lists progressively.

Input Format: Size N and N sorted ints for List 1. Size M and M sorted ints for List 2.

Output Format: One continuous space-separated ordered output representing the merged list nodes.

Constraints: *0 <= N <= 1000. Nodes must be dynamically allocated.*

Hints: *Apply a 'dummy node' approach tracking list heads, recursively pointing next to the currently smallest identified value.*

Test Cases:

Sample Test Case #1

Input:

```
3
5 3 1
3
6 4 2
```

Expected Output:

```
1 2 3 4 5 6
```

Sample Test Case #2

Input:

```
2
10 5
0
```

Expected Output:

```
5 10
```

Sample Test Case #3

Input:

```
0

0
```

Expected Output:

```
Empty
```

Solution Strategy:

O(N+M) merge sort component efficiently loops advancing pointers independently based on strict ordinal evaluation of current node data payload.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* create(int n) { if(n<=0) return NULL; Node *h=NULL, *t=NULL; while(n--) { int v; scanf("%
    d", &v); Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!h) h=t=nn; else { t->n=nn;
     t=nn; } } return h; }
void sort(Node* h) { for(Node* i=h; i; i=i->n) for(Node* j=i->n; j; j=j->n) if(i->d > j->d){ in
    t t=i->d; i->d=j->d; j->d=t; } }
int main() {
    int n,m; if(scanf("%d",&n)!=1) return 0; Node* h1=create(n);
    if(scanf("%d",&m)!=1) return 0; Node* h2=create(m);
    sort(h1); sort(h2);
    Node d; Node* t=&d; d.n=NULL;
    while(h1&&h2){ if(h1->d <= h2->d){ t->n=h1; h1=h1->n; } else { t->n=h2; h2=h2->n; } t=t->n;
     }
    t->n = h1?h1:h2;
    sort(d.n);
    Node* x = d.n;
    if(!x) printf("Empty\n");
    while(x) { printf("%d%c", x->d, x->n?' ':'\n'); x=x->n; }
    return 0;
}
```

Q9. Polynomial Addition

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Perform logical addition algebra combining two polynomials identically.

Input Format: Size N followed by Pairs (coeff, exp). Size M followed by Pairs (coeff, exp).

Output Format: Standard formal polynomial math format e.g., 'Cx^A + Dx^B'.

Constraints: *0 <= N <= 1000. Nodes must be dynamically allocated.*

Hints: *Structurally sequence and sort identically generated polynomials, combining similar exponents, dropping zeros efficiently.*

Test Cases:

Sample Test Case #1

Input:

```
3
5 2 4 1 2 0
3
-5 2 3 1 1 0
```

Expected Output:

```
7x^1 + 3x^0
```

Sample Test Case #2

Input:

```
1
5 2
1
-5 2
```

Expected Output:

```
0
```

Sample Test Case #3

Input:

```
0

1
6 4
```

Expected Output:

```
6x^4
```

Solution Strategy:

Linked structures handle variable algebraic sets natively. Matching exponent values dynamically aggregates coefficients across mathematical inputs directly.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int c, e; struct Node* n; } Node;
Node* create(int k) { Node *h=NULL, *t=NULL; while(k--) { int co, ex; scanf("%d %d", &co, &ex);
     Node* nn=malloc(sizeof(Node)); nn->c=co; nn->e=ex; nn->n=NULL; if(!h) h=t=nn; else { t->n=
    nn; t=nn; } } return h; }
int main() { int n, m; if(scanf("%d", &n)!=1) return 0; Node* p1 = create(n); if(scanf("%d", &m
    )!=1) return 0; Node* p2 = create(m); Node d; Node* t = &d; d.n=NULL; while(p1 && p2) { Nod
    e* nn = malloc(sizeof(Node)); nn->n=NULL; if(p1->e > p2->e) { nn->c = p1->c; nn->e = p1->e;
     p1=p1->n; } else if(p1->e < p2->e) { nn->c = p2->c; nn->e = p2->e; p2=p2->n; } else { nn->
    c = p1->c+p2->c; nn->e = p1->e; p1=p1->n; p2=p2->n; } if(nn->c != 0) { t->n=nn; t=nn; } els
    e free(nn); } while(p1) { Node* nn=malloc(sizeof(Node)); *nn=*p1; nn->n=NULL; t->n=nn; t=nn
    ; p1=p1->n; } while(p2) { Node* nn=malloc(sizeof(Node)); *nn=*p2; nn->n=NULL; t->n=nn; t=nn
    ; p2=p2->n; } if(!d.n) { printf("0\n"); return 0; } Node* x = d.n; while(x) { printf("%dx^%
    d%s", x->c, x->e, x->n?" + ":""); x=x->n; } printf("\n"); return 0; }
```

Week 6: Circular Linked List

Q1. CLL Initialization and Traversal

MEDIUM CODING

Implement a C program utilizing Circular Linked Lists focused on the following tasks: Initialization, complete iteration display, and counting length efficiently.

Input Format: Integer N, N commands (e.g. 'display', 'count', 'ins_end').
Output Format: Integer evaluation or space-separated string representing iterating full cyclic loops.
Constraints: *0 <= N <= 1000. Nodes must circularly map tail to head.*
Hints: *Remember that the final node points to the head pointer instead of strictly resolving to standard NULL.*

Test Cases:

Sample Test Case #1

Input:
```
3
ins_end 10
ins_end 20
display
```
Expected Output:
```
10 20
```

Sample Test Case #2

Input:
```
4
ins_end 5
ins_end 15
count
length
```
Expected Output:
```
2
2
```

Sample Test Case #3

Input:
```
1
display
```
Expected Output:
```
Empty
```

Solution Strategy:

Using standard exact 'do-while' conditions verifies safety alongside bounding pointer memory avoiding infinite while loops on circular architecture.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) {
    Node* nn = malloc(sizeof(Node)); nn->d=v;
    if(!head) { head=nn; head->n=head; }
    else { Node* t=head; while(t->n!=head) t=t->n; nn->n=head; head=nn; t->n=head; }
}
void ie(int v) {
    Node* nn = malloc(sizeof(Node)); nn->d=v;
    if(!head) { head=nn; head->n=head; }
    else { Node* t=head; while(t->n!=head) t=t->n; t->n=nn; nn->n=head; }
}
void ip(int p, int v) {
    if(p==1) { ib(v); return; }
    Node* t=head; for(int i=1; i<p-1 && t->n!=head; i++) t=t->n;
    Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn;
}
void db() {
    if(!head) { printf("Underflow\n"); return; }
    if(head->n==head) { free(head); head=NULL; }
    else { Node* t=head; while(t->n!=head) t=t->n; Node* d=head; head=head->n; t->n=head; free(
    d); }
}
void de() {
    if(!head) { printf("Underflow\n"); return; }
    if(head->n==head) { free(head); head=NULL; }
    else { Node* t=head; while(t->n->n!=head) t=t->n; free(t->n); t->n=head; }
}
void dp(int p) {
    if(!head) { printf("Underflow\n"); return; }
    if(p==1) { db(); return; }
    Node* t=head; for(int i=1; i<p-1 && t->n!=head; i++) t=t->n;
    if(t->n!=head) { Node* d=t->n; t->n=d->n; free(d); }
    else printf("Underflow\n");
}
void dk(int v) {
    if(!head) { printf("Underflow\n"); return; }
    if(head->d==v) { db(); return; }
    Node* t=head; while(t->n!=head && t->n->d!=v) t=t->n;
    if(t->n!=head) { Node* d=t->n; t->n=d->n; free(d); }
    else printf("Not Found\n");
}
void disp() {
    if(!head) { printf("Empty\n"); return; }
    Node* t=head; do { printf("%d%c", t->d, t->n==head?'\n':' '); t=t->n; } while(t!=head);
}
void cnt() {
    if(!head) { printf("0\n"); return; }
    int c=0; Node* t=head; do { c++; t=t->n; } while(t!=head); printf("%d\n", c);
}
void sch(int v) {
    if(!head) { printf("Not Found\n"); return; }
    int p=1; Node* t=head; do { if(t->d==v) { printf("Found at %d\n", p); return; } p++; t=t->n
    ; } while(t!=head);
    printf("Not Found\n");
```

```
}

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
    }
    return 0;
}
```

Q2. Search Operation

MEDIUM CODING

Implement a C program utilizing Circular Linked Lists focused on the following tasks: Searching Elements iteratively.

Input Format: Integer N followed iteratively by specific positional logic requests ('search X').

Output Format: 'Found at P' or explicit precise standard 'Not Found' strings.

Constraints: *0 <= N <= 1000. Nodes must circularly map tail to head.*

Hints: *Iterate tracking index integers, terminating exactly upon reaching original head position cyclically.*

Test Cases:

Sample Test Case #1

Input:

```
4
ins_end 10
ins_end 20
search 20
search 30
```

Expected Output:

```
Found at 2
Not Found
```

Sample Test Case #2

Input:

```
1
search 5
```

Expected Output:

```
Not Found
```

Sample Test Case #3

Input:

```
3
ins_end 1
ins_end 2
search 1
```

Expected Output:

```
Found at 1
```

Solution Strategy:

Circular bounds testing mandates breaking search traversals mathematically immediately following the detection of the origin pointer mapping.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) {
    Node* nn = malloc(sizeof(Node)); nn->d=v;
    if(!head) { head=nn; head->n=head; }
    else { Node* t=head; while(t->n!=head) t=t->n; nn->n=head; head=nn; t->n=head; }
}
void ie(int v) {
    Node* nn = malloc(sizeof(Node)); nn->d=v;
    if(!head) { head=nn; head->n=head; }
    else { Node* t=head; while(t->n!=head) t=t->n; t->n=nn; nn->n=head; }
}
void ip(int p, int v) {
    if(p==1) { ib(v); return; }
    Node* t=head; for(int i=1; i<p-1 && t->n!=head; i++) t=t->n;
    Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn;
}
void db() {
    if(!head) { printf("Underflow\n"); return; }
    if(head->n==head) { free(head); head=NULL; }
    else { Node* t=head; while(t->n!=head) t=t->n; Node* d=head; head=head->n; t->n=head; free(
    d); }
}
void de() {
    if(!head) { printf("Underflow\n"); return; }
    if(head->n==head) { free(head); head=NULL; }
    else { Node* t=head; while(t->n->n!=head) t=t->n; free(t->n); t->n=head; }
}
void dp(int p) {
    if(!head) { printf("Underflow\n"); return; }
    if(p==1) { db(); return; }
    Node* t=head; for(int i=1; i<p-1 && t->n!=head; i++) t=t->n;
    if(t->n!=head) { Node* d=t->n; t->n=d->n; free(d); }
    else printf("Underflow\n");
}
void dk(int v) {
    if(!head) { printf("Underflow\n"); return; }
    if(head->d==v) { db(); return; }
    Node* t=head; while(t->n!=head && t->n->d!=v) t=t->n;
    if(t->n!=head) { Node* d=t->n; t->n=d->n; free(d); }
    else printf("Not Found\n");
}
void disp() {
    if(!head) { printf("Empty\n"); return; }
    Node* t=head; do { printf("%d%c", t->d, t->n==head?'\n':' '); t=t->n; } while(t!=head);
}
void cnt() {
    if(!head) { printf("0\n"); return; }
    int c=0; Node* t=head; do { c++; t=t->n; } while(t!=head); printf("%d\n", c);
}
void sch(int v) {
    if(!head) { printf("Not Found\n"); return; }
    int p=1; Node* t=head; do { if(t->d==v) { printf("Found at %d\n", p); return; } p++; t=t->n
    ; } while(t!=head);
    printf("Not Found\n");
```

```
}

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
    }
    return 0;
}
```

Q3. Insertion Operations

MEDIUM CODING

Implement a C program utilizing Circular Linked Lists focused on the following tasks: Safe structural cyclical dynamic insertions systematically.

Input Format: N operations logically triggering 'ins_beg X', 'ins_end X', 'ins_pos P X'.

Output Format: Explicit verifications are expected upon 'display' commands subsequently.

Constraints: *0 <= N <= 1000. Nodes must circularly map tail to head.*

Hints: *If adjusting head insertion perfectly, remember locating the current tail iterating entire chain re-linking securely to new head.*

Test Cases:

Sample Test Case #1

Input:

```
5
ins_end 10
ins_beg 5
ins_end 20
ins_pos 2 8
display
```

Expected Output:

```
5 8 10 20
```

Sample Test Case #2

Input:

```
2
ins_end 5
display
```

Expected Output:

```
5
```

Sample Test Case #3

Input:

```
3
ins_end 10
ins_pos 1 5
display
```

Expected Output:

```
5 10
```

Solution Strategy:

Maintaining the correct infinite loop structure necessitates O(N) evaluation bounds anytime an operation requires locating the terminal tail block dynamically.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) {
    Node* nn = malloc(sizeof(Node)); nn->d=v;
    if(!head) { head=nn; head->n=head; }
    else { Node* t=head; while(t->n!=head) t=t->n; nn->n=head; head=nn; t->n=head; }
}
void ie(int v) {
    Node* nn = malloc(sizeof(Node)); nn->d=v;
    if(!head) { head=nn; head->n=head; }
    else { Node* t=head; while(t->n!=head) t=t->n; t->n=nn; nn->n=head; }
}
void ip(int p, int v) {
    if(p==1) { ib(v); return; }
    Node* t=head; for(int i=1; i<p-1 && t->n!=head; i++) t=t->n;
    Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn;
}
void db() {
    if(!head) { printf("Underflow\n"); return; }
    if(head->n==head) { free(head); head=NULL; }
    else { Node* t=head; while(t->n!=head) t=t->n; Node* d=head; head=head->n; t->n=head; free(
    d); }
}
void de() {
    if(!head) { printf("Underflow\n"); return; }
    if(head->n==head) { free(head); head=NULL; }
    else { Node* t=head; while(t->n->n!=head) t=t->n; free(t->n); t->n=head; }
}
void dp(int p) {
    if(!head) { printf("Underflow\n"); return; }
    if(p==1) { db(); return; }
    Node* t=head; for(int i=1; i<p-1 && t->n!=head; i++) t=t->n;
    if(t->n!=head) { Node* d=t->n; t->n=d->n; free(d); }
    else printf("Underflow\n");
}
void dk(int v) {
    if(!head) { printf("Underflow\n"); return; }
    if(head->d==v) { db(); return; }
    Node* t=head; while(t->n!=head && t->n->d!=v) t=t->n;
    if(t->n!=head) { Node* d=t->n; t->n=d->n; free(d); }
    else printf("Not Found\n");
}
void disp() {
    if(!head) { printf("Empty\n"); return; }
    Node* t=head; do { printf("%d%c", t->d, t->n==head?'\n':' '); t=t->n; } while(t!=head);
}
void cnt() {
    if(!head) { printf("0\n"); return; }
    int c=0; Node* t=head; do { c++; t=t->n; } while(t!=head); printf("%d\n", c);
}
void sch(int v) {
    if(!head) { printf("Not Found\n"); return; }
    int p=1; Node* t=head; do { if(t->d==v) { printf("Found at %d\n", p); return; } p++; t=t->n
    ; } while(t!=head);
    printf("Not Found\n");
```

```
}

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
    }
    return 0;
}
```

Q4. Deletion Operations

MEDIUM CODING

Implement a C program utilizing Circular Linked Lists focused on the following tasks: Positional parameter Deletions mapping completely correctly.

Input Format: Commands 'del_beg', 'del_end', 'del_key X'.

Output Format: 'Underflow' for empty cyclical states.

Constraints: *0 <= N <= 1000. Nodes must circularly map tail to head.*

Hints: *Deleting strictly single-element chains mandates precise setting of the local core head pointer to NULL unequivocally.*

Test Cases:

Sample Test Case #1

Input:

```
7
ins_end 10
ins_end 20
ins_end 30
del_beg
del_end
del_key 20
display
```

Expected Output:

```
Empty
```

Sample Test Case #2

Input:

```
2
del_beg
display
```

Expected Output:

```
Underflow
Empty
```

Sample Test Case #3

Input:

```
3
ins_end 10
del_pos 1
display
```

Expected Output:

```
Empty
```

Solution Strategy:

Safe cyclic deletion forces mapping precisely to the previous node evaluating structurally correct cyclical pointers re-bounding to standard origins completely.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) {
    Node* nn = malloc(sizeof(Node)); nn->d=v;
    if(!head) { head=nn; head->n=head; }
    else { Node* t=head; while(t->n!=head) t=t->n; nn->n=head; head=nn; t->n=head; }
}
void ie(int v) {
    Node* nn = malloc(sizeof(Node)); nn->d=v;
    if(!head) { head=nn; head->n=head; }
    else { Node* t=head; while(t->n!=head) t=t->n; t->n=nn; nn->n=head; }
}
void ip(int p, int v) {
    if(p==1) { ib(v); return; }
    Node* t=head; for(int i=1; i<p-1 && t->n!=head; i++) t=t->n;
    Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn;
}
void db() {
    if(!head) { printf("Underflow\n"); return; }
    if(head->n==head) { free(head); head=NULL; }
    else { Node* t=head; while(t->n!=head) t=t->n; Node* d=head; head=head->n; t->n=head; free(
    d); }
}
void de() {
    if(!head) { printf("Underflow\n"); return; }
    if(head->n==head) { free(head); head=NULL; }
    else { Node* t=head; while(t->n->n!=head) t=t->n; free(t->n); t->n=head; }
}
void dp(int p) {
    if(!head) { printf("Underflow\n"); return; }
    if(p==1) { db(); return; }
    Node* t=head; for(int i=1; i<p-1 && t->n!=head; i++) t=t->n;
    if(t->n!=head) { Node* d=t->n; t->n=d->n; free(d); }
    else printf("Underflow\n");
}
void dk(int v) {
    if(!head) { printf("Underflow\n"); return; }
    if(head->d==v) { db(); return; }
    Node* t=head; while(t->n!=head && t->n->d!=v) t=t->n;
    if(t->n!=head) { Node* d=t->n; t->n=d->n; free(d); }
    else printf("Not Found\n");
}
void disp() {
    if(!head) { printf("Empty\n"); return; }
    Node* t=head; do { printf("%d%c", t->d, t->n==head?'\n':' '); t=t->n; } while(t!=head);
}
void cnt() {
    if(!head) { printf("0\n"); return; }
    int c=0; Node* t=head; do { c++; t=t->n; } while(t!=head); printf("%d\n", c);
}
void sch(int v) {
    if(!head) { printf("Not Found\n"); return; }
    int p=1; Node* t=head; do { if(t->d==v) { printf("Found at %d\n", p); return; } p++; t=t->n
    ; } while(t!=head);
    printf("Not Found\n");
```

```
}

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "del_key")) { scanf("%d", &v); dk(v); }
        else if(!strcmp(op, "display")) disp();
        else if(!strcmp(op, "count") || !strcmp(op, "length")) cnt();
        else if(!strcmp(op, "search")) { scanf("%d", &v); sch(v); }
    }
    return 0;
}
```

Q5. Circular Validation

MEDIUM CODING

Implement a C program logically strictly diagnosing Linked Lists: Check sequentially if standard Linked List maps dynamically Circular.

Input Format: Integer N, sequential list integers, then a hard boolean C integer explicitly defining cyclical nature globally.

Output Format: 'Circular' definitively or exactly bounded 'Not Circular'.

Constraints: *0 <= N <= 1000. Nodes must circularly map tail to head.*

Hints: *An exhaustive loop terminates when a variable accurately targets head immediately, opposed mathematically to achieving NULL logically.*

Test Cases:

Sample Test Case #1

Input:

```
4
1 2 3 4
1
```

Expected Output:

```
Circular
```

Sample Test Case #2

Input:

```
4
1 2 3 4
0
```

Expected Output:

```
Not Circular
```

Sample Test Case #3

Input:

```
0

0
```

Expected Output:

```
Not Circular
```

Solution Strategy:

Identifying true circular pointer maps sequentially terminates perfectly mapping exact values against historical head pointers directly ensuring complete loop evaluation.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int d; struct Node* n; } Node;
int main() { int n; if(scanf("%d",&n)!=1) return 0; Node *h=NULL, *t=NULL; for(int i=0; i<n; i+
    +){ int v; scanf("%d",&v); Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!h) h=t=n
    n; else{ t->n=nn; t=nn; } } int c; if(scanf("%d",&c)==1 && n>0 && c==1) t->n=h; if(!h) { pr
    intf("Not Circular\n"); return 0; } Node* curr=h->n; while(curr && curr!=h) curr=curr->n; i
    f(curr==h) printf("Circular\n"); else printf("Not Circular\n"); return 0; }
```

Week 7: Double Linked List, Stack, and Queue using Linked List

Q1. Double-Linked List: Insertions and Bidirectional Display

EASY CODING

Implement a **Double-Linked List** evaluating basic insertions (`ins_beg`, `ins_end`, `ins_pos`) and validating bidirectional integrity through exact forward (`display`) and reverse string matching (`rev_display`).

Input Format: Integer N (commands), followed by N operations (e.g. `ins_beg 5`, `rev_display`).

Output Format: Singular space-delimited string of node values from `display` or `rev_display` operations, or `Empty` if no nodes exist.

Constraints: *N <= 1000. Data values within integer limits. Memory limits apply.*

Hints: *Double-Linked Lists require managing `prev` and `next` pointers precisely. For backwards traversal, keep a global `tail` pointer to quickly start from the end.*

Test Cases:

Sample Test Case #1

Input:

```
3
ins_end 10
ins_end 20
display
```

Expected Output:

```
10 20
```

Sample Test Case #2

Input:

```
4
ins_beg 5
ins_beg 1
rev_display
display
```

Expected Output:

```
5 1
1 5
```

Sample Test Case #3

Input:

```
1
display
```

Expected Output:

```
Empty
```

Solution Strategy:

Insertions must strictly update adjoining node logic (both neighbor's `next` and neighbor's `prev`). Printing uses `display` running from global `head`, while `rev_display` executes via iteration starting uniquely backwards from global `tail` pointer.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

Node* head = NULL;
Node* tail = NULL;

void ins_beg(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->prev = NULL;
    nn->next = head;
    if (head) head->prev = nn;
    else tail = nn;
    head = nn;
}

void ins_end(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->next = NULL;
    nn->prev = tail;
    if (tail) tail->next = nn;
    else head = nn;
    tail = nn;
}

void ins_pos(int p, int v) {
    if (p == 1) { ins_beg(v); return; }
    Node* t = head;
    for (int i = 1; i < p - 1 && t; i++) t = t->next;
    if (!t) return;
    if (!t->next) { ins_end(v); return; }
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->prev = t;
    nn->next = t->next;
    t->next->prev = nn;
    t->next = nn;
}

void del_beg() {
    if (!head) { printf("Underflow\n"); return; }
    Node* t = head;
    head = head->next;
    if (head) head->prev = NULL;
    else tail = NULL;
    free(t);
}

void del_end() {
    if (!tail) { printf("Underflow\n"); return; }
    Node* t = tail;
```

```
    tail = tail->prev;
    if (tail) tail->next = NULL;
    else head = NULL;
    free(t);
}

void del_pos(int p) {
    if (!head) { printf("Underflow\n"); return; }
    if (p == 1) { del_beg(); return; }
    Node* t = head;
    for (int i = 1; i < p && t; i++) t = t->next;
    if (!t) { printf("Underflow\n"); return; }
    if (!t->next) { del_end(); return; }
    t->prev->next = t->next;
    t->next->prev = t->prev;
    free(t);
}

void display() {
    if (!head) { printf("Empty\n"); return; }
    Node* t = head;
    while (t) {
        printf("%d%c", t->data, t->next ? ' ' : '\n');
        t = t->next;
    }
}

void rev_display() {
    if (!tail) { printf("Empty\n"); return; }
    Node* t = tail;
    while (t) {
        printf("%d%c", t->data, t->prev ? ' ' : '\n');
        t = t->prev;
    }
}

void search(int v) {
    int p = 1;
    Node* t = head;
    while (t) {
        if (t->data == v) { printf("Found at %d\n", p); return; }
        p++;
        t = t->next;
    }
    printf("Not Found\n");
}

void count() {
    int c = 0;
    Node* t = head;
    while (t) { c++; t = t->next; }
    printf("%d\n", c);
}

int main() {
    int N, v, p;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%s", op);
```

```
        if (!strcmp(op, "ins_beg")) { scanf("%d", &v); ins_beg(v); }
        else if (!strcmp(op, "ins_end")) { scanf("%d", &v); ins_end(v); }
        else if (!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ins_pos(p, v); }
        else if (!strcmp(op, "del_beg")) del_beg();
        else if (!strcmp(op, "del_end")) del_end();
        else if (!strcmp(op, "del_pos")) { scanf("%d", &p); del_pos(p); }
        else if (!strcmp(op, "display")) display();
        else if (!strcmp(op, "rev_display")) rev_display();
        else if (!strcmp(op, "search")) { scanf("%d", &v); search(v); }
        else if (!strcmp(op, "count")) count();
    }
    return 0;
}
```

Q2. Double-Linked List: Arbitrary Deletions

MEDIUM CODING

Advance your DLL implementation to perfectly handle destructive deletion boundaries (`del_beg`, `del_end`, `del_pos`). Validate that deleting unique edge cases updates the global `head` and `tail` correctly without memory access violations.

Input Format: Integer N followed by N newline-separated commands which can be insertion (e.g., "ins_end", "ins_beg" followed by a value) or deletion commands (e.g., "del_*").

Output Format: Space-separated string of node values, or the exact "Underflow" distinct text string on unauthorized deletions.

Constraints: *N <= 1000. All memory spaces gracefully freed mapping memory constraints.*

Hints: *Be profoundly careful to re-assign global `tail = NULL` if deleting the last remaining node in the entire list.*

Test Cases:

Sample Test Case #1

Input:

```
4
ins_end 10
ins_end 20
del_end
display
```

Expected Output:

```
10
```

Sample Test Case #2

Input:

```
5
ins_end 10
ins_end 20
del_beg
rev_display
display
```

Expected Output:

```
20
20
```

Sample Test Case #3

Input:

```
2
del_beg
del_end
```

Expected Output:

```
Underflow
Underflow
```

Solution Strategy:

The logic evaluates bounds check for `Underflow` before updating node adjacencies. By verifying node presence natively prior to pointer unlinking, memory unmapping correctly avoids stray memory leaps or isolated pointer garbage.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

Node* head = NULL;
Node* tail = NULL;

void ins_beg(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->prev = NULL;
    nn->next = head;
    if (head) head->prev = nn;
    else tail = nn;
    head = nn;
}

void ins_end(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->next = NULL;
    nn->prev = tail;
    if (tail) tail->next = nn;
    else head = nn;
    tail = nn;
}

void ins_pos(int p, int v) {
    if (p == 1) { ins_beg(v); return; }
    Node* t = head;
    for (int i = 1; i < p - 1 && t; i++) t = t->next;
    if (!t) return;
    if (!t->next) { ins_end(v); return; }
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->prev = t;
    nn->next = t->next;
    t->next->prev = nn;
    t->next = nn;
}

void del_beg() {
    if (!head) { printf("Underflow\n"); return; }
    Node* t = head;
    head = head->next;
    if (head) head->prev = NULL;
    else tail = NULL;
    free(t);
}

void del_end() {
    if (!tail) { printf("Underflow\n"); return; }
    Node* t = tail;
```

```
    tail = tail->prev;
    if (tail) tail->next = NULL;
    else head = NULL;
    free(t);
}

void del_pos(int p) {
    if (!head) { printf("Underflow\n"); return; }
    if (p == 1) { del_beg(); return; }
    Node* t = head;
    for (int i = 1; i < p && t; i++) t = t->next;
    if (!t) { printf("Underflow\n"); return; }
    if (!t->next) { del_end(); return; }
    t->prev->next = t->next;
    t->next->prev = t->prev;
    free(t);
}

void display() {
    if (!head) { printf("Empty\n"); return; }
    Node* t = head;
    while (t) {
        printf("%d%c", t->data, t->next ? ' ' : '\n');
        t = t->next;
    }
}

void rev_display() {
    if (!tail) { printf("Empty\n"); return; }
    Node* t = tail;
    while (t) {
        printf("%d%c", t->data, t->prev ? ' ' : '\n');
        t = t->prev;
    }
}

void search(int v) {
    int p = 1;
    Node* t = head;
    while (t) {
        if (t->data == v) { printf("Found at %d\n", p); return; }
        p++;
        t = t->next;
    }
    printf("Not Found\n");
}

void count() {
    int c = 0;
    Node* t = head;
    while (t) { c++; t = t->next; }
    printf("%d\n", c);
}

int main() {
    int N, v, p;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%s", op);
```

```
        if (!strcmp(op, "ins_beg")) { scanf("%d", &v); ins_beg(v); }
        else if (!strcmp(op, "ins_end")) { scanf("%d", &v); ins_end(v); }
        else if (!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ins_pos(p, v); }
        else if (!strcmp(op, "del_beg")) del_beg();
        else if (!strcmp(op, "del_end")) del_end();
        else if (!strcmp(op, "del_pos")) { scanf("%d", &p); del_pos(p); }
        else if (!strcmp(op, "display")) display();
        else if (!strcmp(op, "rev_display")) rev_display();
        else if (!strcmp(op, "search")) { scanf("%d", &v); search(v); }
        else if (!strcmp(op, "count")) count();
    }
    return 0;
}
```

Q3. Double-Linked List: Search and Count

EASY CODING

Utilize bidirectional list generation to write custom evaluators counting the sheer volume of elements (`count`) alongside strict linear positional mapping indexing queries (`search`).

Input Format: Integer N followed by mapping commands spanning `search X` querying positions.

Output Format: Single integer counts or dynamically injected text mapped exactly outputting "Found at X" formats.

Constraints: *N <= 1000. Count starts linearly mapping 1-based exact logical indexes.*

Hints: *Both operations are standard `O(n)` tasks traversing explicitly using an independent iteration pointer running through `t=t->next` exclusively avoiding global mutations.*

Test Cases:

Sample Test Case #1

Input:

```
3
ins_end 10
ins_end 20
count
```

Expected Output:

```
2
```

Sample Test Case #2

Input:

```
4
ins_end 10
ins_end 20
ins_end 30
search 20
```

Expected Output:

```
Found at 2
```

Sample Test Case #3

Input:

```
1
count
```

Expected Output:

```
0
```

Solution Strategy:

Iteration executes identically across Single or Double variations mapping list parameters. Validating integer variables linearly identifies key positions avoiding binary or sorted tree lookups in default Linked List mappings.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

Node* head = NULL;
Node* tail = NULL;

void ins_beg(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->prev = NULL;
    nn->next = head;
    if (head) head->prev = nn;
    else tail = nn;
    head = nn;
}

void ins_end(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->next = NULL;
    nn->prev = tail;
    if (tail) tail->next = nn;
    else head = nn;
    tail = nn;
}

void ins_pos(int p, int v) {
    if (p == 1) { ins_beg(v); return; }
    Node* t = head;
    for (int i = 1; i < p - 1 && t; i++) t = t->next;
    if (!t) return;
    if (!t->next) { ins_end(v); return; }
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->prev = t;
    nn->next = t->next;
    t->next->prev = nn;
    t->next = nn;
}

void del_beg() {
    if (!head) { printf("Underflow\n"); return; }
    Node* t = head;
    head = head->next;
    if (head) head->prev = NULL;
    else tail = NULL;
    free(t);
}

void del_end() {
    if (!tail) { printf("Underflow\n"); return; }
    Node* t = tail;
```

```
    tail = tail->prev;
    if (tail) tail->next = NULL;
    else head = NULL;
    free(t);
}

void del_pos(int p) {
    if (!head) { printf("Underflow\n"); return; }
    if (p == 1) { del_beg(); return; }
    Node* t = head;
    for (int i = 1; i < p && t; i++) t = t->next;
    if (!t) { printf("Underflow\n"); return; }
    if (!t->next) { del_end(); return; }
    t->prev->next = t->next;
    t->next->prev = t->prev;
    free(t);
}

void display() {
    if (!head) { printf("Empty\n"); return; }
    Node* t = head;
    while (t) {
        printf("%d%c", t->data, t->next ? ' ' : '\n');
        t = t->next;
    }
}

void rev_display() {
    if (!tail) { printf("Empty\n"); return; }
    Node* t = tail;
    while (t) {
        printf("%d%c", t->data, t->prev ? ' ' : '\n');
        t = t->prev;
    }
}

void search(int v) {
    int p = 1;
    Node* t = head;
    while (t) {
        if (t->data == v) { printf("Found at %d\n", p); return; }
        p++;
        t = t->next;
    }
    printf("Not Found\n");
}

void count() {
    int c = 0;
    Node* t = head;
    while (t) { c++; t = t->next; }
    printf("%d\n", c);
}

int main() {
    int N, v, p;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%s", op);
```

```
        if (!strcmp(op, "ins_beg")) { scanf("%d", &v); ins_beg(v); }
        else if (!strcmp(op, "ins_end")) { scanf("%d", &v); ins_end(v); }
        else if (!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ins_pos(p, v); }
        else if (!strcmp(op, "del_beg")) del_beg();
        else if (!strcmp(op, "del_end")) del_end();
        else if (!strcmp(op, "del_pos")) { scanf("%d", &p); del_pos(p); }
        else if (!strcmp(op, "display")) display();
        else if (!strcmp(op, "rev_display")) rev_display();
        else if (!strcmp(op, "search")) { scanf("%d", &v); search(v); }
        else if (!strcmp(op, "count")) count();
    }
    return 0;
}
```

Q4. Stack Operations using Linked List

MEDIUM CODING

Build a **Stack** dynamically utilizing explicit Linked List principles where items mapping strictly `push`, `pop`, and returning the topmost element seamlessly via `peek` function handlers accurately demonstrate Last-In-First-Out (LIFO) flows.

Input Format: Integer N mapping iterative string blocks calling `push X`, `pop`, `peek`, or `display` stack contents natively.

Output Format: Integer strings evaluating variables returned manually bounding constraints. `Underflow` returned for `pop` issues and `Empty` for missing node visualizations.

Constraints: *N <= 1000. Data values within integer limits. Linked List must dynamically adapt scale memory perfectly.*

Hints: *Visualize a Singly Linked List where you universally insert *AND* delete from exactly the same location (the global `head` pointer, aliased as `top`).*

Test Cases:

Sample Test Case #1

Input:

```
4
push 5
push 10
peek
display
```

Expected Output:

```
10
10 5
```

Sample Test Case #2

Input:

```
2
pop
peek
```

Expected Output:

```
Underflow
Empty
```

Sample Test Case #3

Input:

```
3
push 1
pop
peek
```

Expected Output:

```
Empty
```

Solution Strategy:

LIFO (Last In First Out) operations natively emulate inserting uniformly backwards. By setting `nn->next = top` and `top = nn`, `push` maintains native runtime bounds seamlessly pushing newest data elements highest logically for absolute minimal `O(1)` pointer manipulation sequences.

Reference Solution:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* top = NULL;

void push(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->next = top;
    top = nn;
}

void pop() {
    if (!top) { printf("Underflow\n"); return; }
    Node* t = top;
    top = top->next;
    free(t);
}

void peek() {
    if (!top) { printf("Empty\n"); return; }
    printf("%d\n", top->data);
}

void display() {
    if (!top) { printf("Empty\n"); return; }
    Node* t = top;
    while (t) {
        printf("%d%c", t->data, t->next ? ' ' : '\n');
        t = t->next;
    }
}

int main() {
    int N, v;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%s", op);
        if (!strcmp(op, "push")) { scanf("%d", &v); push(v); }
        else if (!strcmp(op, "pop")) pop();
        else if (!strcmp(op, "peek")) peek();
        else if (!strcmp(op, "display")) display();
    }
    return 0;
}
```

Q5. Queue Operations using Linked List

MEDIUM CODING

Construct a dynamic **Queue** architecture handling iterative node linkages applying First-In-First-Out (FIFO) standards using exact `enqueue` natively linked structures and exact `dequeue` bounds evaluation algorithms alongside raw state visualization strings.

Input Format: Integer N mapped specifically applying logic command routines (`enqueue X`, `dequeue`, `front`, `display`).

Output Format: Printed variables verifying space outputs properly, utilizing exact single string prints ("Empty" or "Underflow") to manage pointer collisions.

Constraints: *N <= 1000. Operations must track bidirectional references gracefully via Dual Pointer Logic.*

Hints: *Keep active logical track of both `front` and `rear` pointers mapping to distinct unique list extremities resolving dynamic insertion memory allocation issues.*

Test Cases:

Sample Test Case #1

Input:

```
4
enqueue 10
enqueue 20
front
display
```

Expected Output:

```
10
10 20
```

Sample Test Case #2

Input:

```
3
enqueue 5
dequeue
front
```

Expected Output:

```
Empty
```

Sample Test Case #3

Input:

```
2
dequeue
display
```

Expected Output:

```
Underflow
Empty
```

Solution Strategy:

A Linked Queue applies elements universally allocating at the tail bounds via tracking explicit `rear->next = nn`

operations avoiding arbitrary list evaluation traversals completely bounding insertion tasks inside raw `O(1)` runtime executions simultaneously alongside native generic `front` bounds node deletions.

Reference Solution:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* front = NULL;
Node* rear = NULL;

void enqueue(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->next = NULL;
    if (rear) rear->next = nn;
    else front = nn;
    rear = nn;
}

void dequeue() {
    if (!front) { printf("Underflow\n"); return; }
    Node* t = front;
    front = front->next;
    if (!front) rear = NULL;
    free(t);
}

void get_front() {
    if (!front) { printf("Empty\n"); return; }
    printf("%d\n", front->data);
}

void display() {
    if (!front) { printf("Empty\n"); return; }
    Node* t = front;
    while (t) {
        printf("%d%c", t->data, t->next ? ' ' : '\n');
        t = t->next;
    }
}

int main() {
    int N, v;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%s", op);
        if (!strcmp(op, "enqueue")) { scanf("%d", &v); enqueue(v); }
        else if (!strcmp(op, "dequeue")) dequeue();
        else if (!strcmp(op, "front")) get_front();
        else if (!strcmp(op, "display")) display();
    }
    return 0;
}
```

Week 9: Priority Queues and DLL Deque

Q1. Priority Queue Linked Implementation

MEDIUM CODING

Implement a **Priority Queue** using a Linked List structure where elements are dynamically ordered based on an Ascending priority value (lower numerical value = higher priority). Ensure equivalent priority numbers respect First-In-First-Out sequences.

Input Format: Integer N mapped specifically applying logic command routines (`enqueue [data] [priority]`, `dequeue`, `peek`, `display`).

Output Format: Printed variables verifying ordered node outputs properly, utilizing exact single string prints ('Empty' or 'Underflow') to manage bounds.

Constraints: *N <= 1000. Data and priority values within standard signed integer limits.*

Hints: *Every explicit enqueue iterates through the linked list to find the first node strictly possessing a lower-placed 'greater' numerical priority, cleanly injecting the new allocation just before it via single pointer replacement.*

Test Cases:

Sample Test Case #1

Input:

```
4
enqueue 10 3
enqueue 20 1
peek
display
```

Expected Output:

```
20
20 10
```

Sample Test Case #2

Input:

```
3
enqueue 5 2
dequeue
peek
```

Expected Output:

```
Empty
```

Sample Test Case #3

Input:

```
2
dequeue
display
```

Expected Output:

```
Underflow
Empty
```

Solution Strategy:

The architecture traverses the active list for every 'enqueue', generating an O(N) ordered insertion schema that guarantees 'dequeue' and 'peek' retain absolute O(1) instantaneous access logic sequentially from local memory, while 'display' requires O(N) traversal.

Reference Solution:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    int priority;
    struct Node* next;
} Node;

Node* front = NULL;

void enqueue(int d, int p) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = d;
    nn->priority = p;
    if (!front || p < front->priority) {
        nn->next = front;
        front = nn;
    } else {
        Node* t = front;
        while (t->next && t->next->priority <= p) t = t->next;
        nn->next = t->next;
        t->next = nn;
    }
}

void dequeue() {
    if (!front) { printf("Underflow\n"); return; }
    Node* t = front;
    front = front->next;
    free(t);
}

void peek() {
    if (!front) { printf("Empty\n"); return; }
    printf("%d\n", front->data);
}

void display() {
    if (!front) { printf("Empty\n"); return; }
    Node* t = front;
    while (t) {
        printf("%d%c", t->data, t->next ? ' ' : '\n');
        t = t->next;
    }
}

int main() {
    int N, d, p;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%s", op);
        if (!strcmp(op, "enqueue")) { scanf("%d %d", &d, &p); enqueue(d, p); }
        else if (!strcmp(op, "dequeue")) dequeue();
        else if (!strcmp(op, "peek")) peek();
        else if (!strcmp(op, "display")) display();
    }
```

```
    return 0;
}
```

Q2. Double-Ended Queue (Deque) using DLL

MEDIUM CODING

Construct a dynamic **Double-Ended Queue (Deque)** perfectly integrated via a Doubly Linked List backbone. Standard routines map insertions (`insert_front`, `insert_rear`) alongside boundaries managing precise endpoint queries.

Input Format: Integer N mapping string operators mapping explicit edge injections (e.g. `delete_rear`, `get_front`).

Output Format: Exact single space list strings executing explicit 'Underflow' boundaries checking dynamic queue nodes.

Constraints: *N <= 1000. Nodes uniformly managed matching precise DLL architecture constraints without segmentation defects.*

Hints: *Be profoundly mindful tracking the 'rear' pointer natively across arbitrary multi-side deletions to maintain accurate LIFO/FIFO hybrid behavior simultaneously.*

Test Cases:

Sample Test Case #1

Input:

```
4
insert_rear 10
insert_front 5
get_front
display
```

Expected Output:

```
5
5 10
```

Sample Test Case #2

Input:

```
3
insert_rear 5
delete_front
get_rear
```

Expected Output:

```
Empty
```

Sample Test Case #3

Input:

```
2
delete_rear
display
```

Expected Output:

```
Underflow
Empty
```

Solution Strategy:

Connecting standard independent DLL memory maps seamlessly to formal Deque functionality enforces generic bounds integrity checks executing natively across 'rear' and 'front' pointers, delivering O(1) performance for insert/delete/get operations, with O(N) for display traversal.

Reference Solution:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

Node* front = NULL;
Node* rear = NULL;

void insert_front(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->prev = NULL;
    nn->next = front;
    if (front) front->prev = nn;
    else rear = nn;
    front = nn;
}

void insert_rear(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->next = NULL;
    nn->prev = rear;
    if (rear) rear->next = nn;
    else front = nn;
    rear = nn;
}

void delete_front() {
    if (!front) { printf("Underflow\n"); return; }
    Node* t = front;
    front = front->next;
    if (front) front->prev = NULL;
    else rear = NULL;
    free(t);
}

void delete_rear() {
    if (!rear) { printf("Underflow\n"); return; }
    Node* t = rear;
    rear = rear->prev;
    if (rear) rear->next = NULL;
    else front = NULL;
    free(t);
}

void get_front() {
    if (!front) printf("Empty\n");
    else printf("%d\n", front->data);
}

void get_rear() {
    if (!rear) printf("Empty\n");
    else printf("%d\n", rear->data);
```

```
}

void display() {
    if (!front) { printf("Empty\n"); return; }
    Node* t = front;
    while (t) {
        printf("%d%c", t->data, t->next ? ' ' : '\n');
        t = t->next;
    }
}

int main() {
    int N, d;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%s", op);
        if (!strcmp(op, "insert_front")) { scanf("%d", &d); insert_front(d); }
        else if (!strcmp(op, "insert_rear")) { scanf("%d", &d); insert_rear(d); }
        else if (!strcmp(op, "delete_front")) delete_front();
        else if (!strcmp(op, "delete_rear")) delete_rear();
        else if (!strcmp(op, "get_front")) get_front();
        else if (!strcmp(op, "get_rear")) get_rear();
        else if (!strcmp(op, "display")) display();
    }
    return 0;
}
```

Week 10: Sorting Techniques

Q1. Merge Sort (Intermediate Steps)

MEDIUM CODING

Implement Merge Sort to sort an array of integers in ascending order. You MUST print the entire array state immediately after every sub-array merge operation finishes.

Input Format: Integer N (size of array), followed by N space-separated integers.

Output Format: Print the full space-separated array on a new line after each successful `merge`.

Constraints: *N <= 1000. Use standard integer typing. Handle an empty list without seg-faulting.*

Hints: *Divide the array into halves, sort them recursively, and merge. Print the global array at the very end of your `merge` function.*

Test Cases:

Sample Test Case #1

Input:

```
5
4 2 5 1 3
```

Expected Output:

```
2 4 5 1 3
2 4 5 1 3
2 4 5 1 3
1 2 3 4 5
```

Sample Test Case #2

Input:

```
3
10 10 10
```

Expected Output:

```
10 10 10
10 10 10
```

Sample Test Case #3

Input:

```
1
5
```

Expected Output:

```
5
```

Solution Strategy:

Merge sort guarantees O(N log N) functionality utilizing implicit tree partitioning. An extra array constructs linearly inside the merge block preventing variable overwriting during swaps.

Reference Solution:

```c
#include <stdio.h>
int n;
void pa(int a[], int n) { for(int i=0; i<n; i++) printf("%d%c", a[i], i==n-1?'\n':' '); }
void merge(int arr[], int l, int m, int r) {
    int i, j, k; int n1 = m - l + 1; int n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; } k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
    pa(arr, n);
}
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
int main() {
    if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    mergeSort(a, 0, n - 1);
    return 0;
}
```

Q2. Heap Sort (Intermediate Steps)

HARD CODING

Implement Heap Sort to sort an array of integers in ascending order. You MUST print the initial fully built max heap array in one line, and then print the array state after EACH element extraction.

Input Format: Integer N (size of array), followed by N space-separated integers.

Output Format: Print the max heap structure first. Then print the array after every max element is bubbled to the end.

Constraints: *N <= 1000. Implement exactly using an array representation heap, O(1) auxiliary space.*

Hints: *Build a Max Heap and print it. Then repeatedly extract the maximum element (root), swap it with the last element of the remaining heap, re-heapify, and PRINT the array.*

Test Cases:

Sample Test Case #1

Input:

```
5
4 2 5 1 3
```

Expected Output:

```
5 3 4 1 2
4 3 2 1 5
3 1 2 4 5
2 1 3 4 5
1 2 3 4 5
```

Sample Test Case #2

Input:

```
3
10 10 10
```

Expected Output:

```
10 10 10
10 10 10
10 10 10
```

Sample Test Case #3

Input:

```
1
5
```

Expected Output:

```
5
```

Solution Strategy:

By reorganizing parent limits recursively against child pointers inside 'heapify', the array implicitly balances like a tree. Iterating extract ops natively results in a pure completely sorted collection.

Reference Solution:

```c
#include <stdio.h>
int n;
void pa(int a[], int n) { for(int i=0; i<n; i++) printf("%d%c", a[i], i==n-1?'\n':' '); }
void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
void heapify(int arr[], int sz, int i) {
    int largest = i; int l = 2 * i + 1; int r = 2 * i + 2;
    if (l < sz && arr[l] > arr[largest]) largest = l;
    if (r < sz && arr[r] > arr[largest]) largest = r;
    if (largest != i) { swap(&arr[i], &arr[largest]); heapify(arr, sz, largest); }
}
void heapSort(int arr[], int sz) {
    for (int i = sz / 2 - 1; i >= 0; i--) heapify(arr, sz, i);
    pa(arr, n);
    for (int i = sz - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
        pa(arr, n);
    }
}
int main() {
    if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    heapSort(a, n);
    return 0;
}
```

Q3. Quick Sort (Intermediate Steps)

MEDIUM **CODING**

Implement Quick Sort to sort an array of integers in ascending order. You MUST print the entire array state immediately after every successful partition operation.

Input Format: Integer N (size of array), followed by N space-separated integers.

Output Format: Print the full space-separated array on a new line after each `partition`.

Constraints: *N <= 1000. The array is updated directly inline with minimal swap overhead variables.*

Hints: *Pick a pivot (last element), partition the array, and IMMEDIATELY print the global array state before recursing into the lower and upper halves.*

Test Cases:

Sample Test Case #1

Input:

```
5
4 2 5 1 3
```

Expected Output:

```
2 1 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Sample Test Case #2

Input:

```
3
10 10 10
```

Expected Output:

```
10 10 10
10 10 10
```

Sample Test Case #3

Input:

```
1
5
```

Expected Output:

```
5
```

Solution Strategy:

Quick sort achieves outstanding functional limits sorting variables completely in-place avoiding external buffer loops. Best case runs highly optimal O(N log N) relying exactly on intelligent partition boundaries.

Reference Solution:

```
#include <stdio.h>
int n;
void pa(int a[], int n) { for(int i=0; i<n; i++) printf("%d%c", a[i], i==n-1?'\n':' '); }
void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) { i++; swap(&arr[i], &arr[j]); }
    }
    swap(&arr[i + 1], &arr[high]);
    pa(arr, n);
    return (i + 1);
}
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main() {
    if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    quickSort(a, 0, n - 1);
    return 0;
}
```

Week 11: Binary Search Tree

Q1. Binary Search Tree Operations

HARD CODING

Construct a robust **Binary Search Tree** tracking insertions (`insert`), deletions (`delete`), indexing queries (`search`), and complete hierarchy traversals (`inorder`, `preorder`, `postorder`). Ensure standard sorting laws dynamically maintain node structures.

Input Format: Integer N (commands), followed by sequential inputs dictating specific operations.

Output Format: Singular space-delimited string of node values dynamically printing trees during traversal routines, or strict text bounding empty searches.

Constraints: *N <= 1000. BST handles variable integer nodes mapping left/right branches appropriately. Memory safely freed inside delete routines.*

Hints: *Inorder traversal uniquely evaluates tree structures logically printing out exact sorted ascending integer sequences natively resolving to purely sorted arrays.*

Test Cases:

Sample Test Case #1

Input:

```
6
insert 10
insert 5
insert 15
inorder
preorder
postorder
```

Expected Output:

```
5 10 15
10 5 15
5 15 10
```

Sample Test Case #2

Input:

```
4
insert 50
insert 30
search 30
search 99
```

Expected Output:

```
Found
Not Found
```

Sample Test Case #3

Input:

```
1
inorder
```

Expected Output:

```
Empty
```

Solution Strategy:

Binary trees dictate left components remaining strictly lesser than the parent structure, and right limits remaining strictly greater. Recursive pointer replacement elegantly evaluates deeper branch loops. Tree traversals (inorder, preorder, postorder) are O(N) because they visit every node, while search, insert, and delete are O(log N) on average for balanced BSTs but can be O(N) in the worst case for skewed trees.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;

Node* root = NULL;
int first = 1;

Node* createNode(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->left = NULL;
    nn->right = NULL;
    return nn;
}

Node* insert(Node* node, int v) {
    if (!node) return createNode(v);
    if (v < node->data) node->left = insert(node->left, v);
    else if (v > node->data) node->right = insert(node->right, v);
    return node;
}

Node* findMin(Node* node) {
    while (node && node->left) node = node->left;
    return node;
}

Node* deleteNode(Node* node, int v) {
    if (!node) return node;
    if (v < node->data) node->left = deleteNode(node->left, v);
    else if (v > node->data) node->right = deleteNode(node->right, v);
    else {
        if (!node->left) {
            Node* temp = node->right;
            free(node);
            return temp;
        } else if (!node->right) {
            Node* temp = node->left;
            free(node);
            return temp;
        }
        Node* temp = findMin(node->right);
        node->data = temp->data;
        node->right = deleteNode(node->right, temp->data);
    }
    return node;
}

void search(Node* node, int v) {
    if (!node) { printf("Not Found\n"); return; }
    if (node->data == v) { printf("Found\n"); return; }
    if (v < node->data) search(node->left, v);
    else search(node->right, v);
```

```
}

void inorder(Node* node) {
    if (node) {
        inorder(node->left);
        if(!first) printf(" ");
        printf("%d", node->data);
        first = 0;
        inorder(node->right);
    }
}

void preorder(Node* node) {
    if (node) {
        if(!first) printf(" ");
        printf("%d", node->data);
        first = 0;
        preorder(node->left);
        preorder(node->right);
    }
}

void postorder(Node* node) {
    if (node) {
        postorder(node->left);
        postorder(node->right);
        if(!first) printf(" ");
        printf("%d", node->data);
        first = 0;
    }
}

void freeTree(Node* node) {
    if (node) {
        freeTree(node->left);
        freeTree(node->right);
        free(node);
    }
}

int main() {
    int N, d;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%19s", op);
        if (!strcmp(op, "insert")) { scanf("%d", &d); root = insert(root, d); }
        else if (!strcmp(op, "delete")) { scanf("%d", &d); root = deleteNode(root, d); }
        else if (!strcmp(op, "search")) { scanf("%d", &d); search(root, d); }
        else if (!strcmp(op, "inorder")) {
            if(!root) printf("Empty\n");
            else { first = 1; inorder(root); printf("\n"); }
        }
        else if (!strcmp(op, "preorder")) {
            if(!root) printf("Empty\n");
            else { first = 1; preorder(root); printf("\n"); }
        }
        else if (!strcmp(op, "postorder")) {
            if(!root) printf("Empty\n");
            else { first = 1; postorder(root); printf("\n"); }
```

```
        }
    }
    freeTree(root);
    return 0;
}
```

Week 12: Graph Traversals

Q1. Breadth First Search (BFS) Traversal

MEDIUM CODING

Perform a Breadth-First Search (BFS) starting from a given source vertex, visiting nodes level-by-level.

Input Format: Integers V (vertices) and E (edges). Followed by E edge pairs 'u v'. Followed by S (starting node).

Output Format: Space-separated list of visited node IDs in the exact visitation order.

Constraints: *V <= 1000. Ensure the queue implementation prevents boundary overwrites. Traversal favors numerically lower node neighbor IDs explicitly on tie breaks.*

Hints: *Use an explicit adjacency matrix and a queue array. Exploration loops should check adjacency matrix columns 0 through V-1 in order so numerically lower neighbors are visited first.*

Test Cases:

Sample Test Case #1

Input:

```
5 4
0 1
0 2
1 3
1 4
0
```

Expected Output:

```
0 1 2 3 4
```

Sample Test Case #2

Input:

```
4 3
0 1
1 2
2 3
0
```

Expected Output:

```
0 1 2 3
```

Sample Test Case #3

Input:

```
5 6
0 1
0 2
1 2
1 3
2 4
3 4
0
```

Expected Output:

```
0 1 2 3 4
```

Solution Strategy:

BFS explores sibling nodes identically across specific depths via classic FIFO execution using a queue. Adjacent numeric checking inside the vertex loop guarantees lower vertices enqueue first naturally resolving branch priority.

Reference Solution:

```c
#include <stdio.h>
#include <stdlib.h>

int adj[1005][1005];
int visited[1005];
int queue[1005];

int main() {
    int V, E;
    if (scanf("%d %d", &V, &E) != 2) return 0;
    for (int i=0; i<E; i++) {
        int u, v;
        if(scanf("%d %d", &u, &v) == 2) {
            if (u >= 0 && u < V && v >= 0 && v < V) {
                adj[u][v] = 1;
                adj[v][u] = 1;
            }
        }
    }
    int S;
    if (scanf("%d", &S) != 1) return 0;
    if (S < 0 || S >= V) return 0;

    int front = 0, rear = 0;

    queue[rear++] = S;
    visited[S] = 1;

    int first = 1;
    while(front < rear) {
        int curr = queue[front++];
        if(!first) printf(" ");
        printf("%d", curr);
        first = 0;

        if (curr >= 0 && curr < V) {
            for(int i=0; i<V; i++) {
                if(adj[curr][i] && !visited[i]) {
                    queue[rear++] = i;
                    visited[i] = 1;
                }
            }
        }
    }
    printf("\n");
    return 0;
}
```

Q2. Depth First Search (DFS) Traversal

MEDIUM CODING

Implement a recursive Depth First Search (DFS) that traverses an unweighted, undirected graph by exploring as deep as possible before backtracking.

Input Format: Integers V (vertices) and E (edges). Followed by E edge pairs 'u v'. Followed by S (starting node).

Output Format: Space-separated sequence depicting exact DFS traversal route explicitly.

Constraints: *V <= 1000; recursion depth must be handled appropriately. Favor lower node ID indexing organically.*

Hints: *Mark a node as visited at the start of its DFS call, then iterate neighbors in increasing ID order and recurse on unvisited neighbors.*

Test Cases:

Sample Test Case #1

Input:

```
5 4
0 1
0 2
1 3
1 4
0
```

Expected Output:

```
0 1 3 4 2
```

Sample Test Case #2

Input:

```
4 3
0 1
1 2
2 3
0
```

Expected Output:

```
0 1 2 3
```

Sample Test Case #3

Input:

```
5 6
0 1
0 2
1 2
1 3
2 4
3 4
0
```

Expected Output:

```
0 1 2 4 3
```

Solution Strategy:

Recursion creates an implicit call stack that traces the traversal path downwards. Tracking a global 'visited' array prevents revisiting nodes and cycles in fully undirected graphs.

Reference Solution:

```c
#include <stdio.h>
#include <stdlib.h>

int visited[1005];
int adj[1005][1005];
int V;
int first = 1;

void dfs(int u) {
    if(!first) printf(" ");
    printf("%d", u);
    first = 0;
    visited[u] = 1;
    for(int i=0; i<V; i++) {
        if(adj[u][i] && !visited[i]) dfs(i);
    }
}

int main() {
    int E;
    if (scanf("%d %d", &V, &E) != 2) return 0;
    for(int i=0; i<V; i++) {
        visited[i] = 0;
        for(int j=0; j<V; j++) adj[i][j] = 0;
    }
    for (int i=0; i<E; i++) {
        int u, v;
        if(scanf("%d %d", &u, &v) == 2) {
            adj[u][v] = 1;
            adj[v][u] = 1;
        }
    }
    int S;
    if (scanf("%d", &S) != 1) return 0;
    dfs(S);
    printf("\n");
    return 0;
}
```

Week 13: Hashing Techniques

Q1. Hashing with Separate Chaining

MEDIUM CODING

Implement a **Hash Table using Separate Chaining** to handle indexing collisions. Maintain bounded array buckets constructed through Linked Lists to permanently store dynamically mapping integers.

Input Format: Integer M (Hash Size) and N (operations). Followed by string commands `insert X`, `search X`, or `display`.

Output Format: Precise outputs reflecting search variables or complete visual states matching `BucketIdx: node->node` structures exactly.

Constraints: *M <= 100. N <= 1000. Elements safely inserted continually without capacity bounds terminating runtime sequences inherently.*

Hints: *Use modulo `data % M` extracting literal index arrays natively tracking independent isolated root nodes inside the master schema.*

Test Cases:

Sample Test Case #1

Input:

```
5 4
insert 10
insert 15
insert 5
display
```

Expected Output:

```
0: 10->15->5
1: Empty
2: Empty
3: Empty
4: Empty
```

Sample Test Case #2

Input:

```
3 3
insert 1
search 1
search 4
```

Expected Output:

```
Found
Not Found
```

Sample Test Case #3

Input:

```
2 1
display
```

Expected Output:

```
0: Empty
1: Empty
```

Solution Strategy:

Separate chaining flawlessly mitigates identical hashing indexes by building explicit Singly Linked architectures inside any collision slot dynamically scaling isolated sizes inherently eliminating bounds limitations externally.

Reference Solution:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* table[1005];
int M = 10;

void insert(int v) {
    int idx = v % M;
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->next = NULL;
    if (!table[idx]) table[idx] = nn;
    else {
        Node* t = table[idx];
        while(t->next) t = t->next;
        t->next = nn;
    }
}

void search(int v) {
    int idx = v % M;
    Node* t = table[idx];
    while(t) {
        if(t->data == v) { printf("Found\n"); return; }
        t = t->next;
    }
    printf("Not Found\n");
}

void display() {
    for(int i=0; i<M; i++) {
        printf("%d: ", i);
        Node* t = table[i];
        if(!t) printf("Empty");
        while(t) {
            printf("%d%s", t->data, t->next ? "->" : "");
            t = t->next;
        }
        printf("\n");
    }
}

int main() {
    int N, v;
    char op[20];
    if (scanf("%d %d", &M, &N) != 2) return 0;
    for(int i=0; i<M; i++) table[i] = NULL;

    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "insert")) { scanf("%d", &v); insert(v); }
        else if(!strcmp(op, "search")) { scanf("%d", &v); search(v); }
        else if(!strcmp(op, "display")) display();
```

```
    }
    return 0;
}
```

Q2. Hashing with Linear Probing

MEDIUM CODING

Build an explicit array-bound **Hash Table modeling Linear Probing** algorithms resolving index loops internally. Iterative checks dynamically seek next open contiguous limits seamlessly shifting inputs appropriately protecting states.

Input Format: Integer M (Hash Size) and N (operations). Followed by explicit indexing calls via `insert X`, `search X`, or `display`.

Output Format: Explicit `Hash Table Full` or index location returns mapping variables checking arrays securely checking empty `-1` defaults.

Constraints: *M <= 100. N <= 1000. Elements are non-negative integers; -1 is reserved as empty marker. Stop gracefully outputting boundaries instead of seg-faulting if input elements overfill explicit structural limits natively.*

Hints: *Linearly search iterative `(idx + i) % M` array locations. Avoid scanning past your starting loop point effectively stopping if total `i == M` size runs empty verifying capacity completely used.*

Test Cases:

Sample Test Case #1

Input:

```
5 4
insert 10
insert 15
insert 5
display
```

Expected Output:

```
0: 10
1: 15
2: 5
3: Empty
4: Empty
```

Sample Test Case #2

Input:

```
3 4
insert 1
insert 1
insert 1
search 1
```

Expected Output:

```
Found at index 1
```

Sample Test Case #3

Input:

```
2 1
display
```

Expected Output:

```
0: Empty
1: Empty
```

Solution Strategy:

Linear probing resolves limits natively allocating inline loops incrementing array indexes utilizing identical mod bounds eliminating all structural link lists entirely relying linearly entirely on existing array gaps.

Reference Solution:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int table[1005];
int M = 10;

void insert(int v) {
    int idx = v % M;
    int i = 0;
    while(i < M && table[(idx + i) % M] != -1) {
        i++;
    }
    if (i == M) {
        printf("Hash Table Full\n");
    } else {
        table[(idx + i) % M] = v;
    }
}

void search(int v) {
    int idx = v % M;
    int i = 0;
    while(i < M && table[(idx + i) % M] != -1) {
        if(table[(idx + i) % M] == v) {
            printf("Found at index %d\n", (idx + i) % M);
            return;
        }
        i++;
    }
    printf("Not Found\n");
}

void display() {
    for(int i=0; i<M; i++) {
        if(table[i] != -1) printf("%d: %d\n", i, table[i]);
        else printf("%d: Empty\n", i);
    }
}

int main() {
    int N, v;
    char op[20];
    if (scanf("%d %d", &M, &N) != 2) return 0;
    for(int i=0; i<M; i++) table[i] = -1;

    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "insert")) { scanf("%d", &v); insert(v); }
        else if(!strcmp(op, "search")) { scanf("%d", &v); search(v); }
        else if(!strcmp(op, "display")) display();
    }
    return 0;
}
```